

A股盯盘助手

产品策划方案

—— Python生态 + 免费部署 + 免费数据源 ——

版本：V1.1（修订版）

日期：2026年3月

技术栈：Streamlit + AkShare + Streamlit Cloud

一、产品概述

1.1 产品定位

A股盯盘助手是一款面向个人投资者的智能股票监控与交易提醒工具，核心功能是帮助投资者实现"高抛低吸"的波段操作策略。产品采用纯Python技术栈开发，基于Streamlit框架构建Web界面，使用AkShare获取免费行情数据，部署在Streamlit Cloud平台上实现公网访问，整个方案零服务器成本。

产品的核心价值主张是"让投资更轻松，让决策更理性"。通过智能化的价格监控和及时的提醒机制，帮助用户克服人性中的贪婪与恐惧，在合适的时机执行合适的操作。系统特别强调"先高抛再低吸"的操作顺序，这符合波段操作的基本逻辑——先在高位卖出锁定利润，再在低位买回降低成本，从而实现持仓成本的持续优化。

1.2 技术方案亮点

本方案的核心亮点在于"零成本部署"和"纯Python生态"。零成本部署意味着用户无需购买云服务器，无需配置域名和SSL证书，只需一个GitHub账号即可将应用部署到公网。纯Python生态意味着整个项目从后端到前端都使用Python语言，降低了学习成本，提高了开发效率，便于后续维护和扩展。

特性	方案	优势
开发语言	纯Python	学习成本低、生态丰富、开发效率高
Web框架	Streamlit	快速构建数据应用、无需前端知识
数据源	AkShare（免费）	无需付费、数据全面、接口稳定
部署平台	Streamlit Cloud（免费）	零服务器成本、自动HTTPS、公网访问
数据存储	本地JSON/SQLite	简单可靠、无需数据库服务器

表1：技术方案亮点

二、技术架构方案

2.1 技术栈选型

技术栈的选型遵循"简单、免费、可靠"的原则。经过对多种方案的对比分析，最终确定采用Streamlit作为Web框架、AkShare作为数据源、Streamlit Cloud作为部署平台的技术组合。这套组合的优势在于：开发效率极高，一个功能完整的MVP版本可以在一周内完成；部署成本为零，Streamlit Cloud提供免费的托管服务；维护成本极低，无需关注服务器运维、数据库管理等问题。

2.1.1 Web框架：Streamlit

Streamlit是一个开源的Python库，专门用于快速构建数据科学和机器学习Web应用。它的核心理念是"用Python写前端"，开发者只需要编写Python代码，无需学习HTML、CSS、JavaScript等前端技术。Streamlit提供了丰富的UI组件，包括文本输入、滑块、下拉框、数据表格、图表等，可以快速构建交互式界面。对于A股盯盘助手这类数据展示和交互为主的应用，Streamlit是理想的选择。

Streamlit的技术特点包括：自动响应式布局，适配不同屏幕尺寸；内置缓存机制，优化数据加载性能；支持热重载，修改代码后自动刷新页面；丰富的第三方组件生态，可以扩展更多功能。Streamlit的缺点是定制性有限，但对于本项目的需求来说已经足够。

2.1.2 数据源：AkShare

AkShare是一个开源的Python金融数据接口库，提供了丰富的A股市场数据获取功能。它支持获取实时行情、历史K线、财务数据、宏观经济数据等多种数据类型。AkShare的数据来源包括东方财富、新浪财经、同花顺等多个公开渠道，数据质量可靠，接口稳定。最重要的是，AkShare完全免费，无需注册账号，无需付费订阅。

AkShare在本项目中的主要用途包括：获取A股实时行情数据（股票代码、当前价、涨跌幅、成交量等）；获取历史K线数据用于技术指标计算；获取股票基本信息（名称、行业、市值等）。AkShare的安装和使用非常简单，只需pip安装后即可调用各种数据接口。

2.1.3 部署平台：Streamlit Cloud

Streamlit Cloud是Streamlit官方提供的免费托管平台，可以将Streamlit应用一键部署到公网。用户只需要将代码推送到GitHub仓库，然后在Streamlit Cloud上关联该仓库，即可自动完成部署。Streamlit Cloud提供免费的计算资源、自动HTTPS证书、自定义域名绑定等功能。对于个人项目和小型应用来说，Streamlit Cloud是完全够用的。

Streamlit Cloud的限制包括：免费版有资源限制（内存1GB、共享CPU）；应用在无访问时会休眠，首次访问需要等待唤醒；不支持后台任务，定时监控需要借助外部服务。针对这些限制，本项目采用"前端触发+本地存储"的方案，用户打开页面时自动刷新数据，无需后台持续运行。

2.2 免费部署方案对比

在确定使用Streamlit Cloud之前，我对多个免费部署平台进行了对比分析。以下是主要平台的对比情况，供参考选择：

平台	免费额度	Python支持	适合场景	推荐指数
Streamlit Cloud	无限制	原生支持	Streamlit应用	★★★★★

Hugging Face Spaces	免费	原生支持	Gradio/Streamlit	★★★★☆
Render	750小时/月	支持	Flask/FastAPI	★★★★☆
Railway	\$5/月额度	支持	通用Python	★★★★☆
PythonAnywhere	有限免费	原生支持	Django/Flask	★★★★☆
Fly.io	3个VM免费	支持	容器化应用	★★★★☆

表2：免费部署平台对比

综合对比后，推荐使用Streamlit Cloud作为首选方案。原因如下：首先，Streamlit Cloud对Streamlit应用有原生支持，部署过程最简单；其次，免费额度无限制，不用担心资源耗尽；第三，官方维护，稳定性和可靠性有保障；第四，自动HTTPS和域名，无需额外配置。如果未来需要更灵活的控制，可以考虑迁移到Render或Railway。

2.3 免费数据源方案

行情数据是本项目的核心依赖，选择合适的数据源至关重要。以下是几个免费的A股数据源对比：

数据源	数据类型	接口方式	稳定性	推荐指数
AkShare	行情/财务/宏观	Python库	高	★★★★★
Tushare	行情/财务	Python库	高	★★★★☆
新浪财经	实时行情	HTTP接口	中	★★★★☆
东方财富	行情/资金流	HTTP接口	中	★★★★☆
腾讯财经	实时行情	HTTP接口	中	★★★★☆

表3：免费数据源对比

推荐使用AkShare作为主要数据源。AkShare的优势在于：接口统一，一个库可以获取多种数据；文档完善，有详细的使用示例；社区活跃，问题能够及时得到解答；持续更新，跟进市场变化。Tushare也是不错的选择，但部分高级数据需要积分，对于本项目来说AkShare已经完全够用。

三、核心功能设计

3.1 功能模块划分

基于Streamlit的特性，本项目的功能模块采用单页面应用的设计方式，通过侧边栏导航切换不同功能区域。主要功能模块包括：标的管理、策略配置、实时监控、提醒记录和系统设置。每个模块独立开发，通过共享的数据文件进行数据交换。

模块	功能描述	Streamlit组件
标的管理	添加/编辑/删除持仓标的	st.form, st.data_editor, st.button
策略配置	设置买卖条件和提醒参数	st.slider, st.number_input, st.selectbox
实时监控	展示实时行情和监控状态	st.metric, st.dataframe, st.line_chart
提醒记录	查看历史提醒和操作记录	st.dataframe, st.expander
系统设置	配置推送渠道和显示偏好	st.checkbox, st.text_input

表4：功能模块与Streamlit组件对应关系

3.2 高抛低吸策略实现

高抛低吸策略的核心是"先卖后买"的顺序控制。系统维护每个标的的状态：初始状态为"等待卖出"，当卖出条件触发后状态变为"等待买入"，买入完成后状态恢复为"等待卖出"。这种状态机设计确保了操作的顺序性，避免用户在未卖出时就进行加仓。

3.2.1 卖出条件判断

卖出条件支持多种设置方式：固定价格卖出（如"股价达到50元时提醒"）、涨幅百分比卖出（如"涨幅超过5%时提醒"）、技术指标卖出（如"KDJ死叉时提醒"）。用户可以选择单一条件或组合条件。系统会实时计算当前价格是否满足条件，满足时触发提醒。

3.2.2 买入条件判断

买入条件同样支持多种设置方式：固定价格买入、跌幅百分比买入、技术指标买入等。关键区别在于，买入条件只有在卖出状态完成后才会生效。系统会记录每次卖出的价格和时间，买入条件可以基于卖出价设置（如"卖出价-3%买入"），也可以基于原始成本价设置。

3.3 提醒推送机制

由于Streamlit Cloud不支持后台任务，提醒推送采用"被动触发"的方式。当用户打开页面时，系统自动检查是否有满足条件的标的，并在页面上显示提醒。同时，系统支持通过邮件发送提醒（使用免费的SMTP服务，如QQ邮箱、163邮箱）。对于更及时的推送需求，可以考虑使用第三方推送服务（如Server酱、Bark等），这些服务提供免费的API接口。

提醒方式	实现方案	成本	及时性
页面提醒	Streamlit st.toast/st.warning	免费	需打开页面
邮件提醒	SMTP（QQ邮箱/163邮箱）	免费	分钟级
微信提醒	Server酱/企业微信机器人	免费	秒级
手机推送	Bark（iOS）/Pushover	免费	秒级

表5：提醒推送方案

四、执行计划

4.1 项目结构设计

项目采用简洁的目录结构，便于开发和维护。主要文件包括：app.py（主程序入口）、pages目录（各功能页面）、lib目录（工具函数和数据操作）、data目录（数据存储）、requirements.txt（依赖列表）。这种结构符合Streamlit的约定，便于在Streamlit Cloud上部署。

项目目录结构如下：app.py是主页面，包含标的列表和监控概览；pages/1_标的管理.py用于添加和管理持仓标的；pages/2_策略配置.py用于设置买卖条件；pages/3_监控面板.py展示实时行情和监控状态；pages/4_提醒记录.py查看历史提醒；lib/stock.py封装股票数据获取逻辑；lib/strategy.py封装策略判断逻辑；lib/notify.py封装推送逻辑；data/stocks.json存储标的配置；data/triggers.json存储触发记录。

4.2 分阶段实施计划

项目开发分为四个阶段，每个阶段有明确的目标和交付物。MVP阶段聚焦核心功能，快速验证产品价值；功能完善阶段补充重要功能，提升产品完整性；体验优化阶段打磨用户界面，提升用户满意度；扩展增强阶段添加高级功能，满足进阶需求。

阶段	周期	核心目标	交付物
MVP	第1-2周	核心功能可用	标的管理+价格监控+页面提醒
功能完善	第3-4周	推送能力完善	邮件/微信推送+策略优化
体验优化	第5-6周	用户体验提升	界面美化+性能优化
扩展增强	第7周起	高级功能	技术指标+数据统计

表6：分阶段实施计划

五、详细执行方案（六步走）

5.1 第一步：环境搭建与项目初始化

开发环境使用Python 3.9+版本，推荐使用conda或venv创建虚拟环境。首先创建项目目录，然后安装必要的依赖包。核心依赖包括：streamlit（Web框架）、akshare（数据源）、pandas（数据处理）、plotly（图表绑定）。安装完成后，创建基础目录结构和配置文件。

具体操作步骤：创建项目目录a_stock_monitor；使用python -m venv venv创建虚拟环境；激活虚拟环境后执行pip install streamlit akshare pandas plotly；创建app.py作为主程序入口；创建pages、lib、data目录；创建requirements.txt记录依赖；创建streamlit/config.toml配置文件。完成环境搭建后，执行streamlit run app.py验证安装成功。

5.2 第二步：标的管理模块开发

标的管理模块负责持仓标的的增删改查。使用Streamlit的st.form组件创建添加表单，使用st.data_editor组件实现编辑功能，使用st.button组件实现删除操作。标的数据存储数据/stocks.json文件中，包含股票代码、股票名称、成本价、持仓数量、添加时间等字段。

开发要点：使用AkShare的stock_info_a_code_name函数获取A股股票列表，实现股票代码的自动补全；使用AkShare的stock_individual_info_em函数获取股票详细信息；使用st.session_state管理表单状态；实现数据的JSON序列化和反序列化；添加数据验证逻辑，确保输入的有效性。

5.3 第三步：策略配置模块开发

策略配置模块负责买卖条件的设置。使用st.slider组件设置价格阈值，使用st.number_input组件设置具体价格，使用st.selectbox组件选择条件类型。策略数据与标的数据关联存储，每个标的可以有多个买卖策略。

策略配置的关键字段包括：标的ID（关联stocks.json）、策略类型（卖出/买入）、条件类型（固定价格/涨幅百分比/技术指标）、条件值（具体数值）、状态（待触发/已触发/已完成）、创建时间、触发时间。系统需要实现策略的CRUD操作，以及策略状态的自动更新。

5.4 第四步：实时监控模块开发

实时监控模块是产品的核心，负责获取实时行情并判断是否触发策略条件。使用AkShare的stock_zh_a_spot_em函数获取A股实时行情数据，该函数返回所有A股的实时行情，速度较快。获取数据后，遍历用户的策略配置，判断当前价格是否满足条件。

监控模块的实现要点：使用st.cache_data装饰器缓存行情数据，避免频繁请求；使用st.rerun或st.autorefresh组件实现页面自动刷新；使用st.metric组件展示关键指标（当前价、涨跌幅、距离目标价等）；使用st.dataframe组件展示监控列表；使用颜色区分不同状态（红色

表示卖出信号、绿色表示买入信号）；触发条件时使用`st.toast`或`st.warning`组件显示提醒。

5.5 第五步：推送模块开发

推送模块负责在条件触发时发送提醒通知。MVP阶段实现页面内提醒，后续迭代添加邮件和微信推送。页面内提醒使用`st.toast`组件实现，在页面右上角显示短暂的通知消息；使用`st.warning`或`st.error`组件在页面上显示持久的提醒信息。

邮件推送的实现方案：使用Python内置的`smtplib`库发送邮件；配置QQ邮箱或163邮箱的SMTP服务（需要在邮箱设置中开启SMTP服务并获取授权码）；创建邮件模板，包含股票名称、当前价格、触发条件、建议操作等信息；在触发条件时调用发送邮件函数。微信推送的实现方案：使用Server酱（免费版每天5条）或企业微信机器人（免费无限制）；调用API接口发送消息；配置推送开关和推送频率。

5.6 第六步：部署上线

部署到Streamlit Cloud的步骤非常简单：首先，在GitHub上创建仓库，将项目代码推送上去；然后，访问`share.streamlit.io`，使用GitHub账号登录；点击"New app"，选择仓库和分支；点击"Deploy"，等待部署完成。部署成功后，系统会分配一个公网访问地址，格式为`https://xxx.streamlit.app`。

部署注意事项：确保`requirements.txt`包含所有依赖；确保`app.py`在仓库根目录；如果使用了敏感信息（如邮箱密码），需要使用`st.secrets`管理，不要硬编码在代码中；Streamlit Cloud会自动检测代码更新并重新部署；如果应用长时间无访问会休眠，首次访问需要等待几秒钟唤醒。

六、成本分析

6.1 开发成本

开发成本主要为时间成本。基于Streamlit框架，一个功能完整的MVP版本可以在1-2周内完成（假设每天投入2-3小时）。相比传统的Web开发方式（需要学习前端技术、配置服务器等），开发效率提升至少3倍。如果选择外包开发，类似功能的开发费用通常在1-3万元。

6.2 运营成本

运营成本为零。Streamlit Cloud提供免费的托管服务，无需购买云服务器；AkShare提供免费的数据接口，无需订阅付费数据源；邮件推送使用免费SMTP服务，微信推送使用免费的Server酱或企业微信机器人。唯一的潜在成本是如果用户量非常大，可能需要升级到Streamlit Cloud的付费版（每月20美元起），但对于个人使用来说免费版完全够用。

项目	传统方案	本方案	节省
----	------	-----	----

云服务器	500-2000元/年	0元	100%
域名	50-100元/年	0元	100%
SSL证书	0-500元/年	0元	100%
数据源	1000-5000元/年	0元	100%
推送服务	100-500元/年	0元	100%
合计	1650-8100元/年	0元	100%

表7：成本对比分析

七、后续扩展方向

7.1 功能扩展

在MVP版本稳定运行后，可以考虑以下功能扩展：技术指标策略支持，包括MA均线、MACD、KDJ、RSI、布林带等常用指标；策略回测功能，验证策略在历史数据上的表现；组合管理功能，支持多标的的统一管理和资产配置分析；数据统计功能，包括收益统计、操作复盘、策略效果评估等。

7.2 技术升级

如果用户量增长或功能需求增加，可以考虑以下技术升级：数据存储从JSON文件迁移到SQLite或云数据库（如Supabase免费版）；部署平台从Streamlit Cloud迁移到Render或Railway，获得更多控制权；添加后台任务支持，使用APScheduler或外部定时服务（如cron-job.org）实现真正的实时监控；添加用户系统，支持多用户独立使用。